

Red Hat Data Grid 8.6

Using the RESP protocol endpoint with Data Grid

Use the Data Grid RESP endpoint to interact with remote caches

Last Updated: 2026-06-03

Red Hat Data Grid 8.6 Using the RESP protocol endpoint with Data Grid

Use the Data Grid RESP endpoint to interact with remote caches

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux ® is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack ® Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

The RESP endpoint allows Redis clients to connect to one or more Data Grid backed RESP servers and perform cache operations.

Table of Contents

RED HAT DATA GRID 3

DATA GRID DOCUMENTATION 4

DATA GRID DOWNLOADS 5

MAKING OPEN SOURCE MORE INCLUSIVE 6

CHAPTER 1. DATA GRID RESP ENDPOINT 7
1.1. CONFIGURING CACHES FOR THE RESP ENDPOINT 7
Explicit configuration for cache 7
Explicit RESP endpoint configuration 8
Clustered configuration 9
1.2. MAPPING CACHES TO REDIS LOGICAL DATABASES 10
1.3. DIFFERENCES BETWEEN DATA GRID AND REDIS IMPLEMENTATION 10
1.3.1. Protocol version 10
1.3.2. Isolation 10
1.3.3. Transactions 10
1.3.4. Lua Scripts 11
1.4. REDIS COMMANDS 11

RED HAT DATA GRID

Data Grid is a high-performance, distributed in-memory data store.

Schemaless data structure

Flexibility to store different objects as key-value pairs.

Grid-based data storage

Designed to distribute and replicate data across clusters.

Elastic scaling

Dynamically adjust the number of nodes to meet demand without service disruption.

Data interoperability

Store, retrieve, and query data in the grid from different endpoints.

DATA GRID DOCUMENTATION

Documentation for Data Grid is available on the Red Hat customer portal.

- Data Grid 8.6 Documentation
- Data Grid 8.6 Component Details
- Supported Configurations for Data Grid 8.6
- Data Grid 8 Feature Support
- Data Grid Deprecated Features and Functionality

DATA GRID DOWNLOADS

Access the Data Grid Software Downloads on the Red Hat customer portal.



NOTE

You must have a Red Hat account to access and download Data Grid software.

MAKING OPEN SOURCE MORE INCLUSIVE

Red Hat is committed to replacing problematic language in our code, documentation, and web properties. We are beginning with these four terms: master, slave, blacklist, and whitelist. Because of the enormity of this endeavor, these changes will be implemented gradually over several upcoming releases. For more details, see our CTO Chris Wright's message.

CHAPTER 1. DATA GRID RESP ENDPOINT

The RESP endpoint is enabled by default on the single-port endpoint. Redis client connections will automatically be detected and routed to the internal connector. The RESP endpoint works with:

- Standalone Data Grid Server deployments, exactly like standalone Redis, where each server instance runs independently of each other.
- Clustered Data Grid Server deployments, where server instances replicate or distribute data between each other. Clustered deployments provides clients with failover capabilities.

Prerequisites

- Install Data Grid Server.

Procedure

1. Create a user

Verification

When you start Data Grid Server check for the following log message:

```
[org.infinispan.SERVER] ISPN080018: Started connector Resp (internal)
```

You can now connect to the RESP endpoint with a Redis client. For example, with the Redis CLI you can do the following to add an entry to the cache:

```
redis-cli -p 11222 --user username --pass password
```

```
127.0.0.1:11222> SET k v
OK
127.0.0.1:11222> GET k
"v"
127.0.0.1:11222> quit
```

1.1. CONFIGURING CACHES FOR THE RESP ENDPOINT

The RESP endpoint automatically configures and starts a **respCache** cache. This cache has the following configuration:

- **local-cache** or **distributed-cache** depending on the Data Grid Server clustering mode.
- **application/octet-stream** encoding for both keys and values.
- **RESPHashFunctionPartitioner** hash partitioner, which supports the CRC16 hashing used by Redis clients

Explicit configuration for cache

It is possible to supply a custom configuration for the cache, as long as it does not violate the requirements of the RESP connector, in which case the server will raise an exception and will not start. Main constraints are:

- hash partitioning function must be **org.infinispan.distribution.ch.impl.RESPHashFunctionPartitioner**.
- key encoding must be **application/octet-stream**.

Example of explicit cache configuration follows.

XML

```
<distributed-cache name="respCache" aliases="0" owners="2"
                   key-partitioner="org.infinispan.distribution.ch.impl.RESPHashFunctionPartitioner"
                   mode="SYNC" remote-timeout="17500" statistics="true">
 <encoding media-type="application/octet-stream"/>
</distributed-cache>
```

JSON

```
{
  "respCache": {
    "distributed-cache": {
      "aliases": ["0"],
      "owners": "2",
      "key-partitioner": "org.infinispan.distribution.ch.impl.RESPHashFunctionPartitioner",
      "mode": "SYNC",
      "statistics": true,
      "encoding": {
        "media-type": "application/octet-stream"
      }
    }
  }
}
```

YAML

```
respCache:
  distributedCache:
    aliases:
      - "0"
    owners: "2"
    keyPartitioner: "org.infinispan.distribution.ch.impl.RESPHashFunctionPartitioner"
    mode: "SYNC"
    statistics: "true"
    encoding:
      mediaType: "application/octet-stream"
```

TIP

Configure your cache value encoding with Protobuf encoding if you want to view cache entries in the Data Grid Console (**value media-type="application/x-protostream"**).

Explicit RESP endpoint configuration

If the implicit configuration used by the single-port endpoint does not fit your needs, explicit configuration is available.

XML

```xml
<endpoints>
  <endpoint socket-binding="default" security-realm="default">
    <resp-connector cache="mycache" />
    <hotrod-connector />
    <rest-connector/>
  </endpoint>
</endpoints>
```

JSON

```
{
  "server": {
    "endpoints": {
      "endpoint": {
        "socket-binding": "default",
        "security-realm": "default",
        "resp-connector": {
          "cache": "mycache"
        },
        "hotrod-connector": {},
        "rest-connector": {}
      }
    }
  }
}
```

YAML

```
server:
  endpoints:
    endpoint:
      socketBinding: "default"
      securityRealm: "default"
      respConnector:
        cache: "mycache"
      hotrodConnector: ~
      restConnector: ~
```

Clustered configuration

Data Grid is a horizontally scalable Resp-compatible server with high availability capabilities. Data Grid provides clustering mechanisms with automatic failover detection and membership discovery by default. Topology changes are handled automatically, and adding or removing nodes to the cluster is a streamlined process that automatically redistributes entries between nodes without downtime. The cluster is homogeneous, where nodes are the primary and the backup owners of entries, guaranteeing that entries are available even in case of server failures.

Backed by a consistent hash, compatible with the hash-slot algorithm in Resp clients, entries are distributed across cluster members. Data Grid automatically redirects requests to the proper entry

owner. As a result, hash tags in keys and the **-MOVED** error response for requests are not required even for multi-key operations accessing keys with different owners.

TIP

Data Grid allows multiple logical databases with the SELECT command in clustering mode.

1.2. MAPPING CACHES TO REDIS LOGICAL DATABASES

Use the cache **aliases** configuration attributes to map caches to Redis logical databases. The default **respCache** is mapped to logical database **0**.

TIP

Data Grid can use multiple logical databases even in clustered mode, as opposed to Redis which only supports database **0** when using Redis Cluster.

1.3. DIFFERENCES BETWEEN DATA GRID AND REDIS IMPLEMENTATION

Data Grid provides different guarantees compared to Redis for some functionalities. This chapter summarizes the differences and, when possible, alternative configurations for a similar behavior.

1.3.1. Protocol version

Data Grid only supports RESP version 3. Attempting to use version 2, will result in an error.

1.3.2. Isolation

Redis utilizes a single thread to handle user requests, which provides serializable isolation and atomic behavior for multi-key requests. Data Grid provides a relaxed isolation level, which is configurable. Concurrent requests might perceive a partial result for commands that access multiple keys, such as **MSET**, where only a subset of the keys were inserted in the cache before the operation finishes.

The only alternative in Data Grid for an atomic behavior is to submit multi-keys operations within a **MULTI...EXEC** block. However, this requires configuring the default cache with transactional capabilities.



WARNING

Data Grid does not provide serializable transactions. See the cache configuration guide for more information.

1.3.3. Transactions

Applications utilizing the **MULTI...EXEC** commands should update the default cache configuration to enable transactional capabilities. Transactions in Redis do not have the concept of rollback to revert in case of failures. Data Grid provides ACID transactions and rollback in case of failures. Additionally, the

transactions are distributed in cluster mode and can operate across many slots.

TIP

The recommended configuration utilizes **PESSIMISTIC** locking and transaction mode different from **NONE**. See the cache configuration guide for more information.

1.3.4. Lua Scripts

Lua scripts executed by Data Grid serialize responses using the RESP3 protocol. This is equivalent to invoking **redis.setresp(3)** in every script.

1.4. REDIS COMMANDS

The Data Grid RESP endpoint implements the following Redis commands:

- APPEND
- AUTH
- BLPOP
- BLMPOP
- BRPOP
- CLIENT GETNAME
- CLIENT ID
- CLIENT INFO
- CLIENT LIST
- CLIENT SETINFO
- CLIENT SETNAME
- COMMAND
- CONFIG
- CLUSTER KEYSLOT
- CLUSTER NODES



NOTE

This commands includes all required fields, but some fields are set to **0** as they do not apply to Data Grid.

- CLUSTER SHARDS
- CLUSTER SLOTS

- DBSIZE
- DECR
- DECRBY
- DEL
- DISCARD
- ECHO
- EVAL



NOTE

unlike Redis, which maintains a bounded cache of ephemeral scripts, Data Grid discards the script immediately after execution.

- EVAL_RO



NOTE

unlike Redis, which maintains a bounded cache of ephemeral scripts, Data Grid discards the script immediately after execution.

- EVALSHA
- EVALSHA_RO
- EXEC
  See the MULTI command.
- EXISTS
- EXPIRE
- EXPIREAT
- EXPIRETIME
- FLUSHALL



NOTE

This command behaves like **FLUSHDB** and flushes only the current database.

- FLUSHDB
- GET
- GETDEL
- GETEX

- GETRANGE
- GETSET



NOTE

This command is deprecated. Use the **SET** command with the appropriate flags instead.

- HDEL
- HELLO



NOTE

Only protocol version 3 (RESP3) is supported.

- HEXISTS
- HGET
- HGETALL
- HINCRBY
- HINCRBYFLOAT
- HKEYS
- HLEN
- HMGET
- HMSET
- HRANDFIELD
- HSCAN
- HSET
- HSETNX
- HSTRLEN
- HVALS
- INCR
- INCRBY
- INCRBYFLOAT
- INFO



NOTE

This implementation attempts to return all attributes that a real Redis server returns. However, in most cases, the values are set to **0** because they cannot be retrieved, or don't apply to Data Grid.

- JSON.ARRAPPEND
- JSON.ARRINDEX
- JSON.ARRINSERT
- JSON.ARRLEN
- JSON.ARRTRIM
- JSON.CLEAR
- JSON.ARRPOP
- JSON.DEBUG MEMORY
- JSON.DEL
- JSON.FORGET
- JSON.GET



NOTE

> JSON filter with RegEx differencies from Redis implementation:

1. Pattern could be expressed in Redis style "(i)pattern" or Java style "/.**pattern.**/i)".
2. Dynamic filter with regex is not supported. i.e. **$.store.book[?(@.name =~ $.pattern)]** doesn't work.
3. Filtering an array returns all matching elements and all nested arrays containing matching elements.

- link:https://redis.io/docs/latest/commands/json.merge [JSON.MERGE]
- JSON.MSET

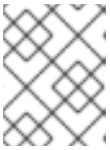


NOTE

This command is non atomic

- JSON.MGET
- JSON.NUMINCRBY
- JSON.NUMMULTBY

- JSON.OBJKEYS
- JSON.OBJLEN
- JSON.RESP
- JSON.SET
- JSON.STRAPPEND
- JSON.STRLEN
- JSON.TOGGLE
- JSON.TYPE
- KEYS
- LINDEX
- LINSERT



NOTE

The current implementation has a time complexity of O(N), where N is the size of the list.

- LLEN
- LCS
- LMOVE



NOTE

The current implementation is atomic for rotation when the source and destination are the same list. For different lists, there is relaxed consistency for concurrent operations or failures unless the resp cache is configured to use transactions.

- LMPOP
- LPOP
- LPOS
- LPUSH
- LPUSHX
- LRANGE
- LREM
- LSET

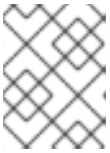
- LTRIM
- MEMORY USAGE



NOTE

This command will return the memory used by the key and the value. It doesn't include the memory used by additional metadata associated with the entry.

- MEMORY STATS



NOTE

This command will return the same fields as a real Redis server, but all values will be set to **0**.

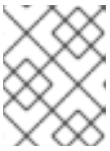
- MGET
- MODULE LIST



NOTE

This command always returns an empty list of modules.

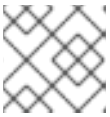
- MSET
- MSETNX
- MULTI



NOTE

The current implementation has a relaxed isolation level. Redis offers serializable transactions.

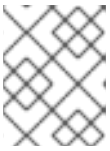
- PERSIST
- PFADD
- PEXPIRE
- PEXPIREAT
- PEXPIRETIME
- PING
- PSETEX



NOTE

This command is deprecated. Use the **SET** command with the appropriate flags.

- PSUBSCRIBE
- PUBSUB CHANNELS
- PUBSUB NUMPAT
- PTTL
- PUBLISH
- PUNSUBSCRIBE
- QUIT
- RANDOMKEY
- RPOP
- RPOPLPUSH
- RPUSH
- RPUSHX
- READONLY
- READWRITE
- RENAME
- RENAMENX
- RESET
- SADD
- SCARD
- SCAN



NOTE

Cursors are reaped in case they have not been used within a timeout. The timeout is 5 minutes.

- SCRIPT EXISTS
- SCRIPT FLUSH
- SCRIPT LOAD
- SDIFF
- SDIFFSTORE
- SELECT



NOTE

Data Grid allows the SELECT command both in local and clustered mode, unlike Redis Cluster which forbids use of this command and only supports database zero.

- SET
- SETEX



NOTE

This command is deprecated. Use the **SET** command with the appropriate flags instead.

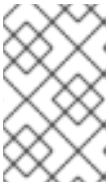
- SETNX



NOTE

This command is deprecated. Use the **SET** command with the appropriate flags instead.

- SETRANGE
- SINTER
- SINTERCARD
- SINTERSTORE
- SISMEMBER
- SORT
- SORT_RO
- SMEMBERS
- SMISMEMBER
- SMOVE

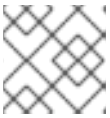


NOTE

The current implementation has a relaxed isolation level. A client can see the source and destination set without the element. The operation is not atomic, it could remove the element from source and fails to insert to the target set.

- SPOP
- SRANDMEMBER
- SREM

- SSCAN
- STRALGO LCS
- STRLEN
- SUBSTR



NOTE

This command is deprecated. Use the **GETRANGE** command instead.

- SUBSCRIBE
- SUNION
- SUNIONSTORE
- TIME
- TOUCH
- TTL
- TYPE
- UNSUBSCRIBE
- UNWATCH
- WATCH
- ZADD
- ZCARD
- ZCOUNT
- ZDIFF
- ZDIFFSTORE
- ZINCRBY
- ZINTER
- ZINTERCARD
- ZINTERSTORE
- ZLEXCOUNT
- ZMPOP
- ZMSCORE
- ZPOPMAX

- ZPOPMIN
- ZUNION
- ZUNIONSTORE
- ZRANDMEMBER
- ZRANGE
- ZRANGEBYLEX
- ZRANGEBYSCORE
- ZREVRANGE
- ZREVRANGEBYLEX
- ZREVRANGEBYSCORE
- ZRANGESTORE
- ZRANK
- ZREM
- ZREMRANGEBYLEX
- ZREMRANGEBYRANK
- ZREMRANGEBYSCORE
- ZREVRANK
- ZSCAN
- ZSCORE